

Experimental Setup

Experimental Setup I

This section details the controlled vocabulary, worked examples, and software environment used to produce the results.

Controlled vocabulary I

The DSL's vocabulary is declared once in code and consulted by every gate and the compiler — never re-declared per method:

Controlled vocabulary I

Module	Declares	Cardinality
<code>src/methods_dsl/vocabulary.py</code>	<code>StepKind</code> , <code>Target</code> , <code>target_accepts</code>	9 step kinds, 3 targets
<code>src/methods_dsl/units.py</code>	<code>Dimension</code> , <code>Quantity</code> , the unit table	18 controlled units across 8 dimensions
<code>src/methods_dsl/validation.py</code>	The four staged gates	4 gates, fixed order

Worked examples I

`all_example_methods()` (`src/methods_dsl/examples_methods.py`) returns 2 methods:

Worked examples I

Method	Domain	Target	Notable structure
PBSPreparation	Wet-lab bench preparation (BPL's own domain; an original example)	HUMAN	A strict 5-step linear chain with a final VALIDATE step
SensorCalibrationSweep	Instrument calibration (a non-biology controlled procedure)	AUTOMATED	Mixed automated MEASURE/COMPUTE steps and a HUMAN ANNOTATE sign-off step

Worked examples I

The second example exists specifically to demonstrate that the controlled vocabulary generalizes beyond wet-lab protocols, as sec. 2 claims.

Pipeline conditions I

The experiment overlay (`experiment_plan.yaml`) declares three conditions:

Pipeline conditions I

- ▶ **declared_method** (reference) — a method constructed directly as Python objects, before any validation gate has run.
- ▶ **validated_method** (proposed) — the same method after passing all four staged gates and deterministic compilation to a scheduled Plan.
- ▶ **automated_target_variant** (variant) — PBSPreparation's steps recompiled against an AUTOMATED execution target, ablating the target-compatibility gate's HUMAN/AUTOMATED step boundary (a HUMAN method's steps must all be HUMAN-compatible; an AUTOMATED method's steps may be either).

Pipeline conditions I

The primary metric is gate pass rate: the fraction of staged-gate evaluations a method's steps satisfy.

Computational environment I

Computational environment I

- ▶ **Language:** Python 3.12.12 on Darwin arm64 (see root `pyproject.toml` for the supported version range).
- ▶ **Core dependencies:** `pyyaml`, `matplotlib` (declared in `domain_profile.yaml::required_packages`); the DSL library itself (`src/methods_dsl/`) has zero third-party dependencies beyond the standard library, with one declared infrastructure logging exception (`_logging.py`).
- ▶ **Headless plotting:** the analysis script sets `MPLBACKEND=Agg` before importing `matplotlib`.

Pipeline ordering I

The typical analysis order is:

Pipeline ordering I

1. `scripts/methods_analysis.py` — compiles every example method, runs all gates, exports worklist/CSV/Mermaid/JSON per method, demonstrates the provenance hash-chain, and writes `output/figures/step_counts.png`, printing each output path for manifest collection.
2. `scripts/z_generate_manuscript_variables.py` — reads `manuscript/config.yaml` and the analysis outputs, then resolves every generated variable in `manuscript/*.md`.
3. PDF rendering reads the resolved manuscript tree so figure paths and prose match the analysis that just completed.

Relation to results I

Relation to results I

Result (sec. 4)	Producing function (src/methods_dsl/)	Primary inputs
Compiled-plan summary	<code>compile_method()</code>	<code>all_example_methods()</code>
Step-count figure	<code>len(plan.steps)</code> per method	<code>output/data/compiled_plans.json</code>
Gate pass tally	<code>run_all_gates()</code>	Each example method
Determinism check	<code>compile_method()</code> called twice	<code>all_example_methods()</code>
Trust-chain verification	<code>append_record()</code> / <code>verify_chain()</code>	Demonstration chain in <code>scripts/methods_analysis.py</code>

Relation to results I

This table is descriptive documentation only; it is not executed as code during the build.